Plan de Desarrollo - Dijkstra Optimizado

Implementacion utilizando PriorityQueue en lugar de busqueda lineal sobre un array.

Objetivo

Implementar una version optimizada del algoritmo de Dijkstra para obtener caminos minimos en grafos con pesos no negativos. La optimizacion principal consiste en utilizar una PriorityQueue (min-heap) para seleccionar eficientemente el proximo nodo con menor distancia conocida.

Etapa 1 - Representacion del grafo

Definir la estructura del grafo mediante listas de adyacencia. Cada arista debe almacenar destino y peso. Antes de implementar Dijkstra, probar que sea posible agregar nodos, agregar aristas y consultar los vecinos de un nodo.

Etapa 2 - Estructuras auxiliares

Crear las estructuras necesarias para el algoritmo: distancias, predecesores y PriorityQueue. La cola debe almacenar pares (nodo, distancia) y ordenar por menor distancia.

Etapa 3 - Inicializacion

Inicializar todas las distancias en infinito, todos los predecesores en null y asignar distancia 0 al nodo origen. Luego insertar el origen en la PriorityQueue.

Etapa 4 - Extraccion del minimo

Implementar el ciclo principal. Mientras la PriorityQueue no este vacia, extraer con poll() el nodo con menor distancia. Esta operacion reemplaza la busqueda lineal del minimo de la version con array.

Etapa 5 - Relajacion de aristas

Para cada vecino del nodo actual, calcular nuevaDistancia = dist[actual] + peso(actual, vecino). Si la nueva distancia es menor que la conocida, actualizar distancia y predecesor, e insertar la nueva entrada en la PriorityQueue.

Etapa 6 - Entradas repetidas en la PriorityQueue

Puede haber varias entradas del mismo nodo. No es necesario eliminarlas. Cuando se extrae una entrada, si su distancia es mayor que la distancia actualmente registrada para ese nodo, se ignora por estar desactualizada.

Etapa 7 - Integracion del algoritmo completo

Unir inicializacion, extraccion del minimo, relajacion y descarte de entradas viejas. Verificar el resultado sobre grafos pequenos donde el camino minimo pueda calcularse manualmente.

Etapa 8 - Reconstruccion del camino

Usar el arreglo o mapa de predecesores para reconstruir el camino minimo entre origen y destino. Se parte del destino, se siguen predecesores hasta el origen y luego se invierte la secuencia.

Etapa 9 - Tests

Agregar pruebas para: grafo simple, varios caminos posibles, nodo inaccesible, actualizacion de una distancia ya descubierta, entradas repetidas en la cola y reconstruccion correcta del camino.

Etapa 10 - Analisis de complejidad

Comparar la version original con array frente a la optimizada. Con array, la seleccion del minimo lleva a una complejidad aproximada O(V^2). Con listas de adyacencia y PriorityQueue, la complejidad pasa a O((V + E) log V), usualmente expresada como O(E log V).

Pseudocodigo propuesto

```
DijkstraOptimizado(G, origen):

    para cada vertice v:
        dist[v] = infinito
        prev[v] = null

    dist[origen] = 0
    pq.add((0, origen))

    mientras pq no este vacia:
        (distU, u) = pq.poll()

        si distU > dist[u]:
            continuar

        para cada arista (u, v) de peso w:
            nuevaDist = dist[u] + w

            si nuevaDist < dist[v]:
                dist[v] = nuevaDist
                prev[v] = u
                pq.add((nuevaDist, v))

    retornar dist, prev
```

Orden de trabajo sugerido

Ticket	Objetivo
1	Crear Graph y Edge
2	Implementar lista de adyacencia
3	Crear QueueEntry para PriorityQueue
4	Probar orden por menor distancia
5	Inicializar distancias y predecesores
6	Implementar relajacion individual
7	Implementar ciclo completo de Dijkstra
8	Manejar entradas viejas de la PriorityQueue
9	Reconstruir el camino minimo
10	Agregar tests y documentar complejidad

Condicion importante: Dijkstra requiere que los pesos de las aristas sean no negativos.